

Report for Object Detection with TensorFlow Lite on JPG
Images

Student Name : **L.Sharon Danish Laus**

Student ID : **25249347**

Revision Timestamp: 03/11/2025 23:33:18

Contents

Introduction	2
Results Table.....	3
Screenshots.....	3
Model Performance Analysis	5
Critical Findings	5
Speed-Accuracy Trade-Off	6
Recommendation.....	6
Conclusion	6
References	6
Figure: All Detections.....	3

Introduction:

Object detection is a computer vision challenge that is used to find and locate various objects in images by depositing bounding boxes and confidence measures to the detection. As edge computing emerges, there is an increasing need in effective deep learning models, which can be used to conduct real-time object detection on resource-restricted machines such as the Raspberry Pi. TensorFlow Lite overcomes this issue by offering optimized light neural networks which are optimized for embedded systems.

The laboratory experiment that compares the performance of three pre-trained TensorFlow Lite object detection models on a Raspberry Pi 4 is the default model (SSD MobileNet v2), SSD MobileNet v1, and EfficientDet-Lite0. The experimental approach will include experimental capturing of the test images of five different objects standing still and the measurement of inference time, confidence rating, and detection reliability using the confusion matrix analysis.

This report will provide a systematic comparison of the model performance, examine speed-accuracy trade-offs and also come up with evidence-based recommendations on the choice of the best model to use in real-time object detection on edge devices. The results are relevant to the knowledge of the interaction between various neural network configurations and computational performance and detection accuracy in real embedded computing systems.

Results Table:

Objects	Person	Mouse	Remote	Scissors	Bed
Default Model	88%, 303.42 ms , TP	70%,329.35ms,TP	55%,323.22ms, TP	59%,307.39ms , TP	Not Detected ,FN
Mobile Net	80%,115.10ms , TP	75%, 114.27ms , TP	Not Detected , FN	Not Detected FN	75%,114.93ms TP
Efficientdet	85%, 259.68 ms , TP	75%,290.31ms , TP	Not Detected ,FN	58%,262.29ms , TP	62%,259.68ms , TP

There was a case where efficient det detected the bed the person was sitting on as couch which can be classified as false positive.

Screenshots:

Left most picture (Default model) , Middle picture (Mobile Net) , Right most picture (Efficientdet)

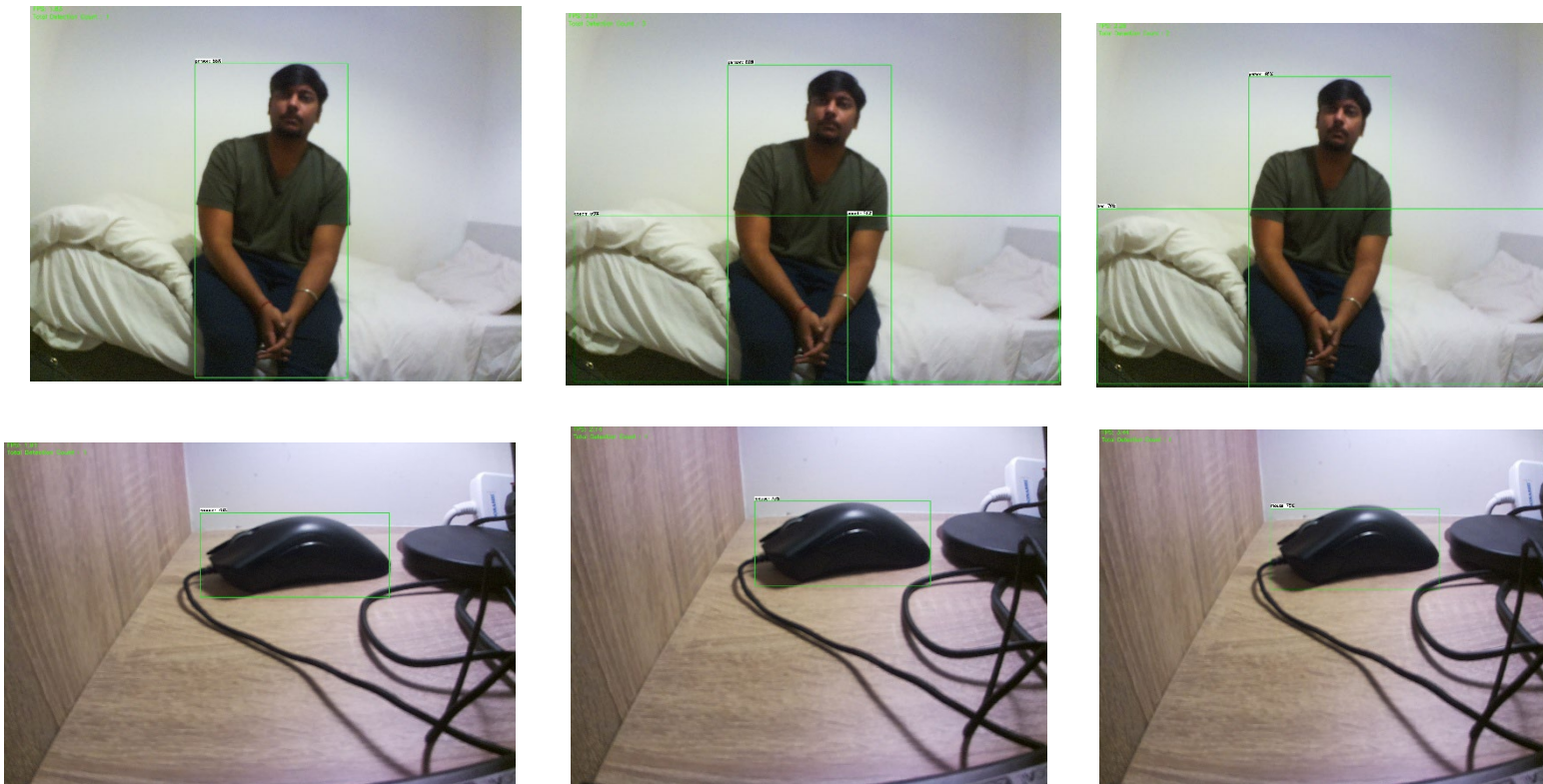
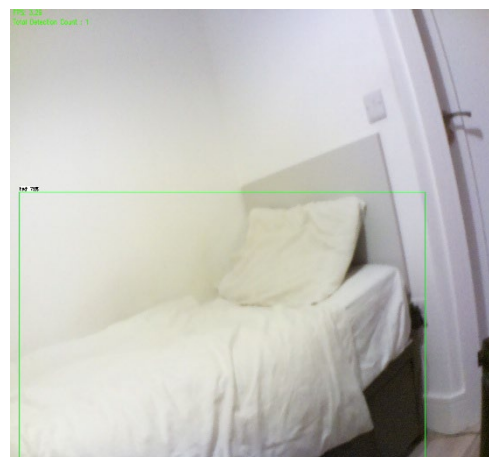
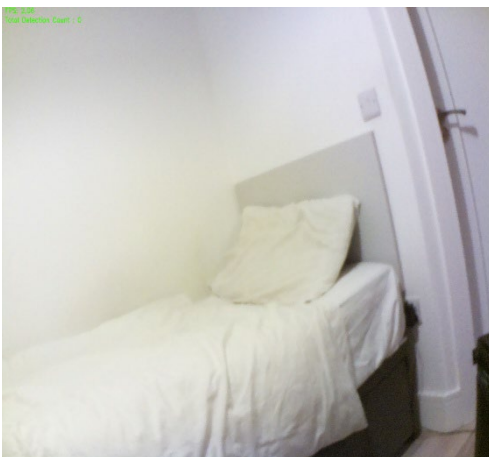
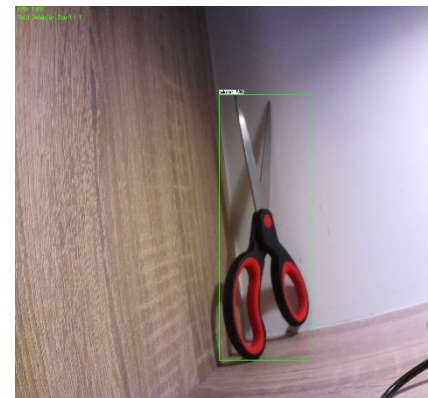
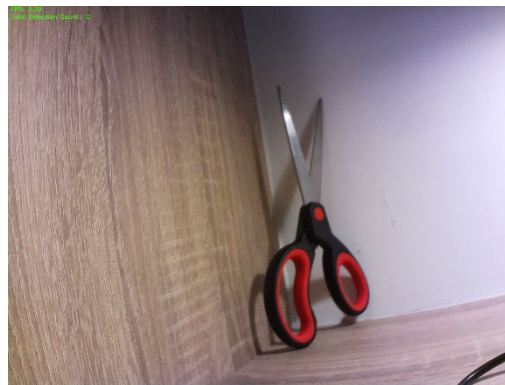
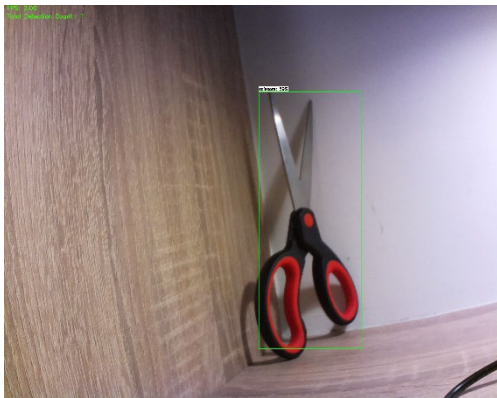
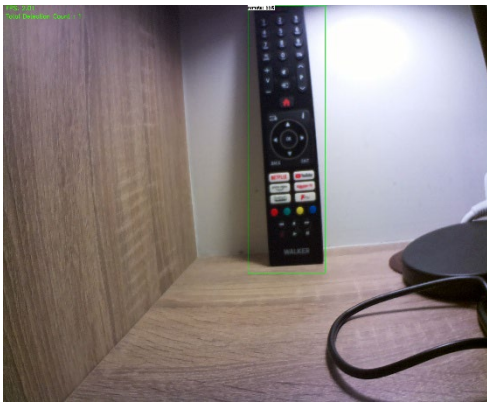


Figure : All Detections

Object Detection with TensorFlow Lite on JPG Images



Model Performance Analysis:

Confusion matrix (TP, FP, FN, TN) was used to assess three TensorFlow Lite models on five objects: person, bed, mouse, scissors and remote.

Personal Model Performance:

Default Model

Best rate of detection of 4/5 objects correctly identified (80% accuracy). True Positives: person (88%), mouse (70%), remote (55%), scissors (59%). False Positives: None. False Negatives: Bed undetected. Mean: 315.85ms (3.2 FPS). The model was very precise with none of the cases being misclassified.

Mobile Net:

Quickest inference (114.77ms) (8.7 FPS) with the lowest accuracy (40%). True Positives: person (80%), mouse (75%). False Positives: Bed misaligned couch (75 percent confidence) when individual was sitting on it- this error due to occlusions demonstrates what is known to be problematic with compressed models dealing with partial occlusions. False Negatives: Remote, scissors undetected. Speed was sacrificed greatly at the expense of accuracy.

EfficientDet:

Balanced at 60 percent accuracy (3/5 objects) and inference time of 267.99ms (3.7 FPS). True Positives - person (85%), mouse (75%), scissors (58%); False Positives: None. False Negatives: Remote and bed undetected. Was able to detect scissors successfully where Mobile Net could not and demonstrated superior ability to detect small objects.

Critical Findings:

Occlusion Challenge: Bed-to-couch misclassification of Mobile Net happened in case of a person sitting on the bed, which proved to be vulnerable to the occlusion cases typical of the indoor setting. Both Default Model and EfficientDet avoided this mistake, which is more evidence of superior performance of compressed models compared to standard models.

Small Object Detection: Remote (55) and scissors (58) were difficult. only default Model identified remote, EfficientDet only scissors. Mobile Net also missed the two implying aggressive compression at the expense of small-object detection.

Consistent Detection Person (80-88% confidence) All models had consistent detection, indicating that large objects can be detected in architectures.

Speed-Accuracy Trade-Off:

The 114.77ms of Mobile Net can be used to process in real time with 40% accuracy, and critical misclassification makes it impractical. The 315.85ms of Default Model is less but 80 percent accuracy with no false positives gives the Model better reliability. EfficientDet has middle ground of 267.99ms and accuracy of 60%.

Recommendation:

Select Default Model (SSD MobileNet v2) to be used in Raspberry Pi applications that need a stable recognition. Although the inference time is 315.85ms, with an 80% accuracy, zero false hits, and the widest object coverage (4/5 objects, as well as challenging remote) it is the most reliable. False positives cause an inappropriate reaction of a system within an application such as home automation or assistive robotics, so precision is important.

Alternative: EfficientDet fits in scenarios that can withstand 60% accuracy of 17% faster (267.99ms). It has zero false positives and ability to detect scissors which gives good precision.

Do NOT use Mobile Net unless a speed of less than 120ms is absolutely imperative, and it is acceptable that the application will suffer both reduced rate of detection and both false and true classification of furniture in human-occupied areas.

Conclusion:

None of the models can perform optimally on Raspberry PI 4 on all metrics. The high 80% accuracy and no false positive of Default Model are better in reliability sensitive applications, even though its speed is slow at 3.2 FPS. The special occlusion-incurred bed/ couch misclassification of Mobile Net indicates a severe weakness of compressed models: a lack of resistance to real-world situations. This is an example of the bias-variance trade-off in machine learning Mobile Net has a simplified architecture that causes systematic classification errors in difficult detection situations.

References:

- 1) A. Priyadarshan, "TensorFlow 2 Lite Object Detection on the Raspberry Pi," GitHub repository, 2022, <https://github.com/armaanpriyadarshan/TensorFlow-2-Lite-Object-Detection-on-the-Raspberry-Pi>.

Object Detection with TensorFlow Lite on JPG Images

- 2) Google, "SSD MobileNet v1 TensorFlow Lite Model," TensorFlow Hub, Kaggle, <https://www.kaggle.com/models/tensorflow/ssd-mobilenet-v1/tfLite>
- 3) Google, "EfficientDet-Lite TensorFlow Lite Model," TensorFlow Hub, Kaggle, <https://www.kaggle.com/models/tensorflow/efficientdet/tfLite>.
- 4) CS6461 Lab5 - Object Detection with TensorFlow Lite on JPG Images